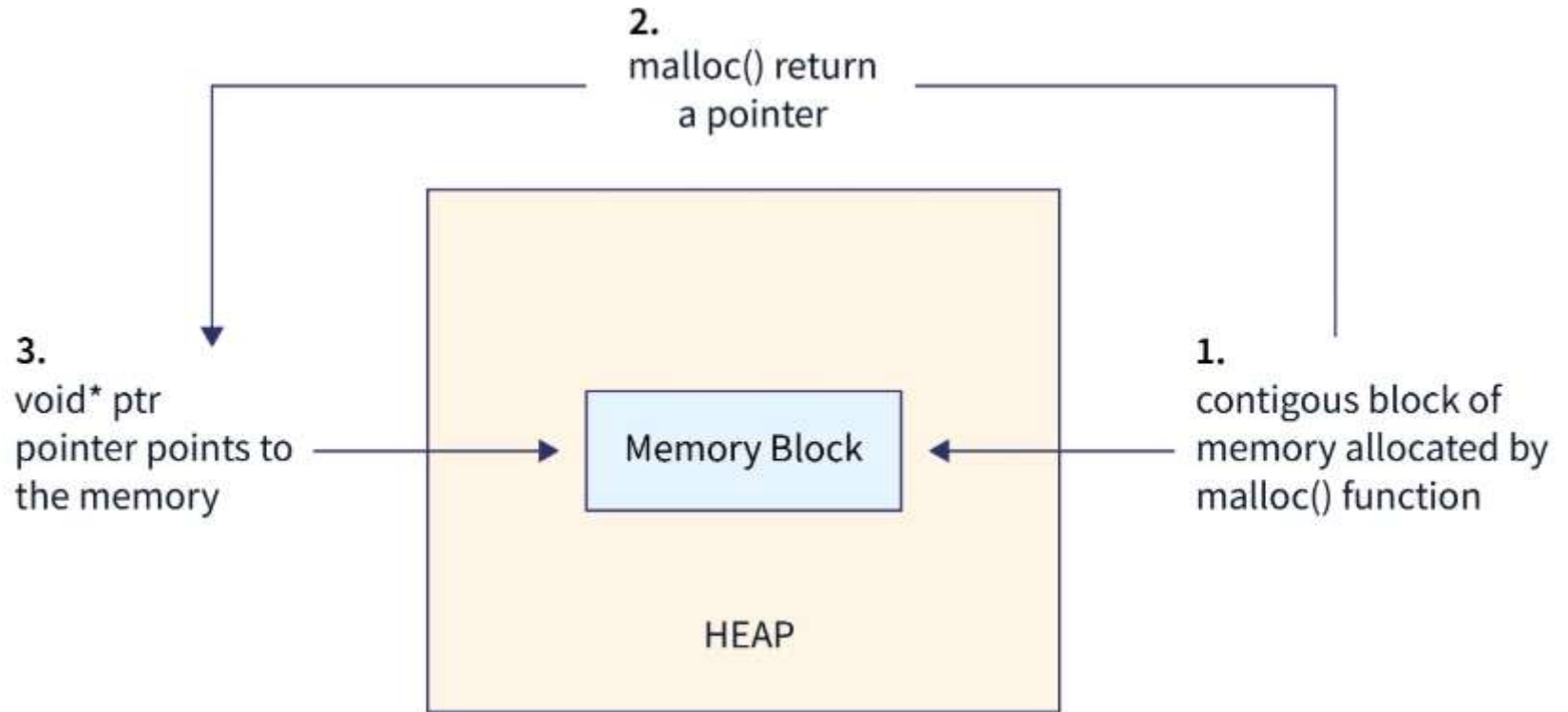


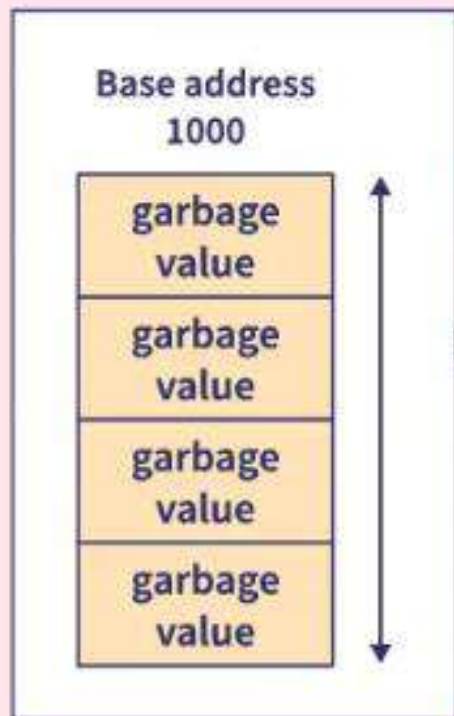
Module-1 Additional Information

Difference between malloc() and calloc() :



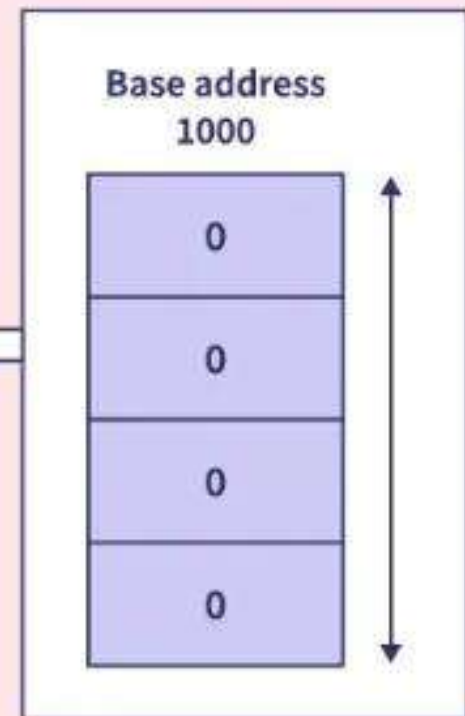
Difference between malloc() and calloc() :

malloc



VS

calloc



```
malloc(5 * sizeof(int))
```

```
+-----+-----+-----+-----+-----+
|  ??  |  ??  |  ??  |  ??  |  ??  |
+-----+-----+-----+-----+-----+
```

```
calloc(5, sizeof(int))
```

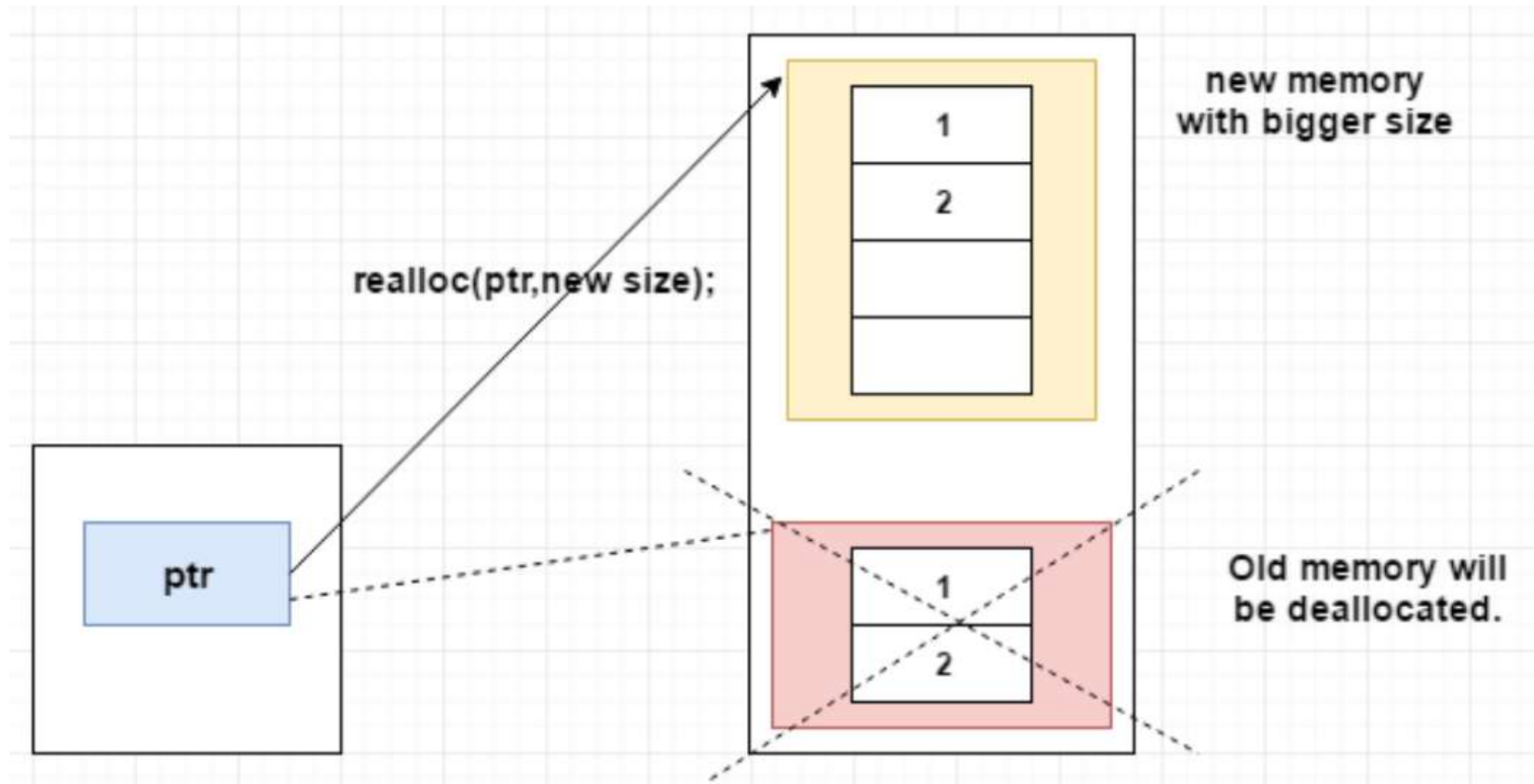
```
+-----+-----+-----+-----+-----+
|  0   |  0   |  0   |  0   |  0   |
+-----+-----+-----+-----+-----+
```



Quick Comparison Table (Exam-friendly)

Feature	malloc()	calloc()
Full form	Memory Allocation	Contiguous Allocation
Initialization	✗ No (garbage)	✓ Yes (0)
Arguments	1	2
Speed	Faster	Slightly slower
Memory layout	Single block	Contiguous blocks

realloc()



Case 1: Expanding memory (space available nearby)

Before `realloc()`

```
sql  
  
malloc(3 * sizeof(int))
```

```
+-----+  
| 10 | 20 | 30 |  
+-----+
```

After `realloc(ptr, 5 * sizeof(int))`

```
diff  
  
+-----+  
| 10 | 20 | 30 | ?? | ?? |  
+-----+
```

- ✓ Old data is **preserved**
- ✓ Extra memory is **uninitialized**

Case 2: Expanding memory (no adjacent space → new location)

Old Block

```
+-----+  
| 10 | 20 | 30 |  
+-----+  
^ old freed
```

New Block

```
+-----+  
| 10 | 20 | 30 | ?? | ?? |  
+-----+
```

- ✓ Data is **copied**
- ✓ Old memory is **freed automatically**
- ✓ Pointer may **change address**

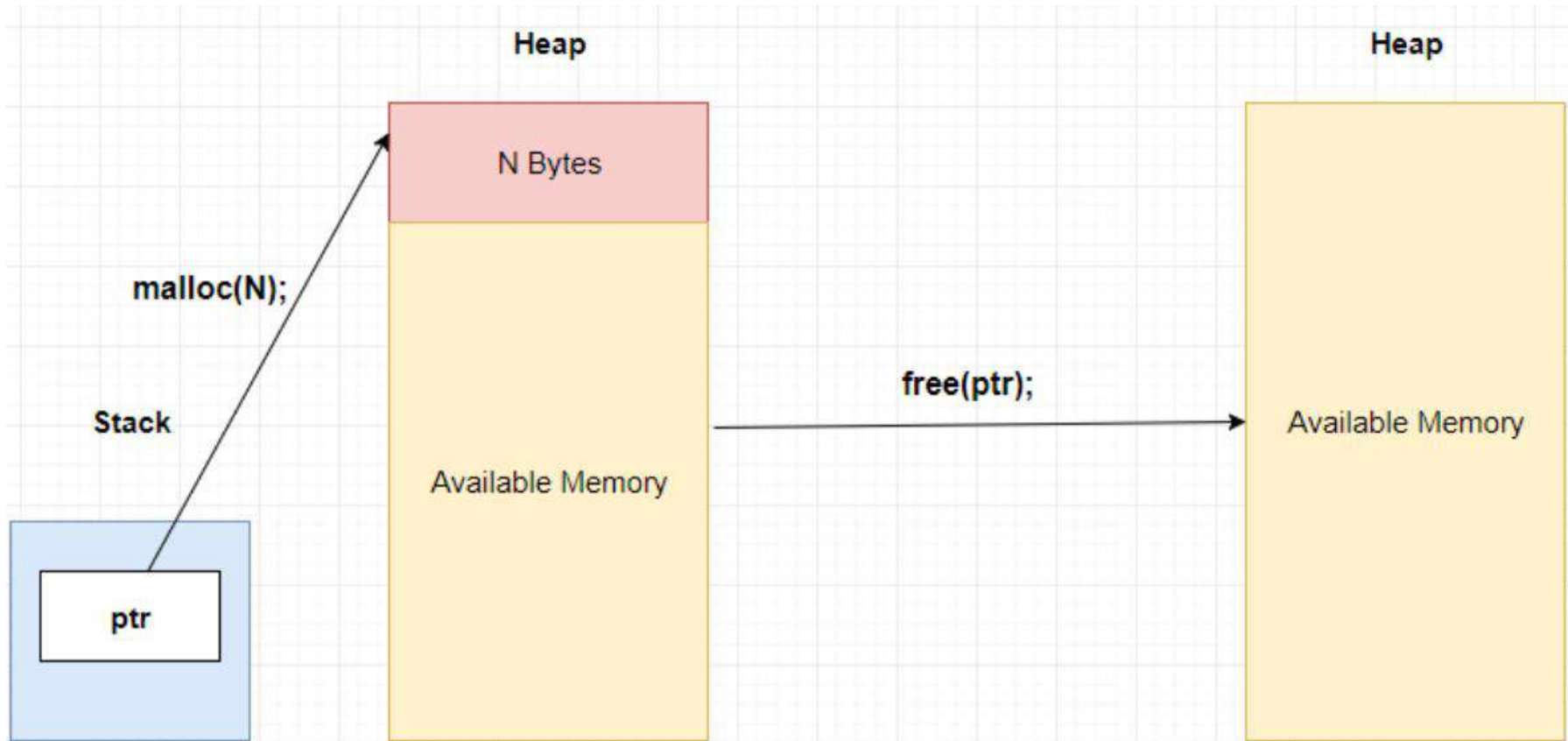
Case 3: Shrinking memory

```
realloc(ptr, 2 * sizeof(int))
```

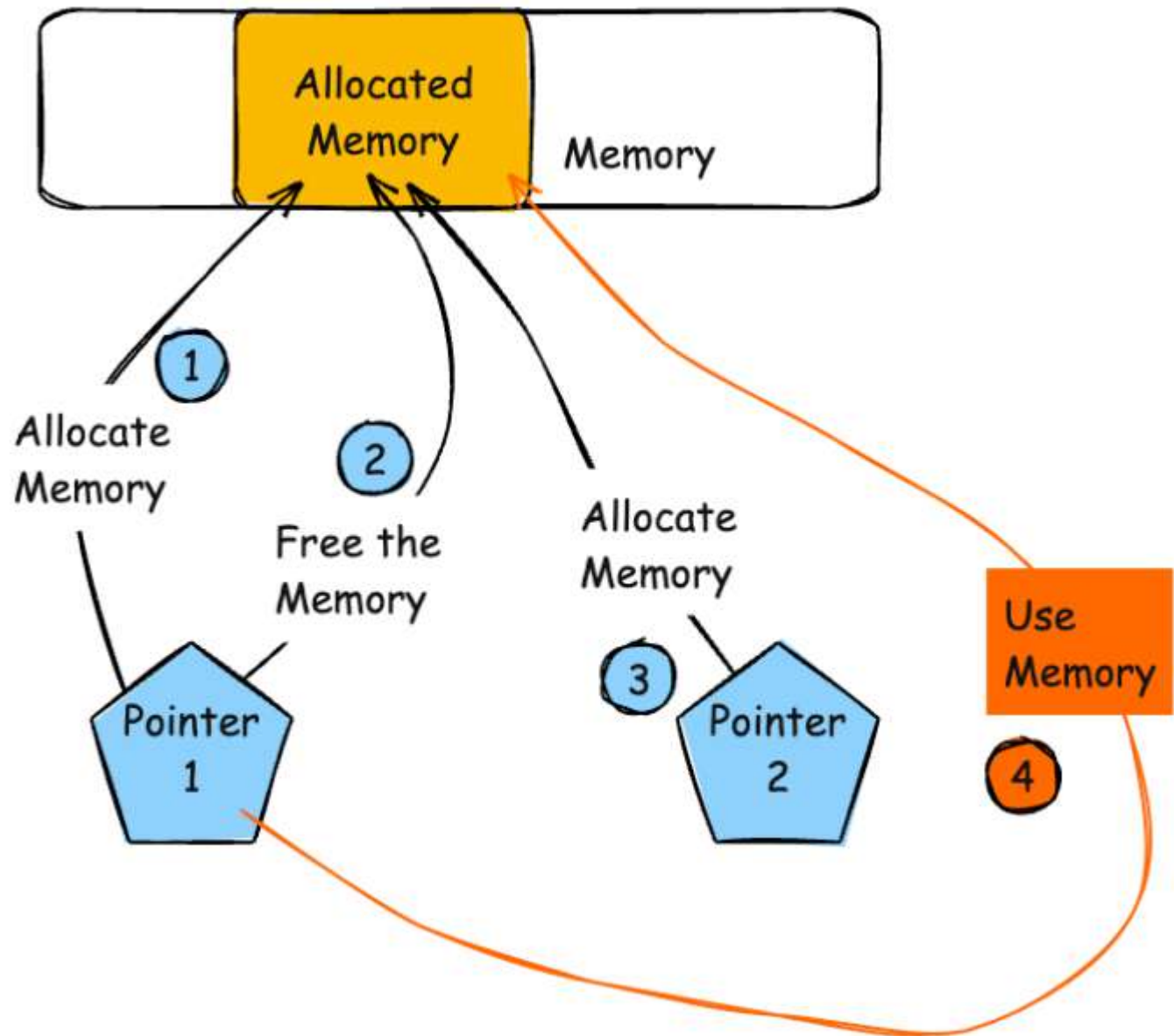
```
+---+---+  
| 10 | 20 |  
+---+---+
```

- ✓ Extra memory is released
- ✓ Initial elements are retained

`free()`



free()



free()

```
free(ptr);
```

ptr

|



+---+---+---+---+---+

| freed memory |

+---+---+---+---+---+

- ✓ Memory returned to **heap**
- ✗ Pointer becomes a **dangling pointer**

//C program for PRACTICE PROBLEM-1

```
#include <stdio.h>
```

```
int main() {
```

```
    char c = 'A';
```

```
    int i = 10;
```

```
    float f = 3.14f;
```

```
    double d = 9.876;
```

```
    /* Pointer initialization */
```

```
    char *pc = &c;
```

```
    int *pi = &i;
```

```
    float *pf = &f;
```

```
    double *pd = &d;
```

```
    /* Printing size of pointers */
```

```
    printf("Size of char pointer   : %zu bytes\n", sizeof(pc));
```

```
    printf("Size of int pointer    : %zu bytes\n", sizeof(pi));
```

```
    printf("Size of float pointer   : %zu bytes\n", sizeof(pf));
```

```
    printf("Size of double pointer  : %zu bytes\n", sizeof(pd));
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
//C program for PRACTICE PROBLEM-2
```

```
int main() {
```

```
    /* Array initialization */
```

```
    char cArr[5] = {'A', 'B', 'C', 'D', 'E'};
```

```
    int iArr[5] = {1, 2, 3, 4, 5};
```

```
    float fArr[5] = {1.1, 2.2, 3.3, 4.4, 5.5};
```

```
    double dArr[5] = {10.1, 20.2, 30.3, 40.4, 50.5};
```

```
    /* Pointer to array */
```

```
    char (*pc)[5] = &cArr;
```

```
    int (*pi)[5] = &iArr;
```

```
    float (*pf)[5] = &fArr;
```

```
    double (*pd)[5] = &dArr;
```

```
    /* Size of pointers to arrays */
```

```
    printf("Size of pointer to char array : %zu bytes\n", sizeof(pc));
```

```
    printf("Size of pointer to int array : %zu bytes\n", sizeof(pi));
```

```
    printf("Size of pointer to float array : %zu bytes\n", sizeof(pf));
```

```
    printf("Size of pointer to double array : %zu bytes\n\n", sizeof(pd));
```

```
/* Size of arrays */  
printf("Size of char array  : %zu bytes\n", sizeof(cArr));  
printf("Size of int array   : %zu bytes\n", sizeof(iArr));  
printf("Size of float array  : %zu bytes\n", sizeof(fArr));  
printf("Size of double array : %zu bytes\n", sizeof(dArr));  
  
return 0;  
}
```

//C program to Insert Element to End of the LIST

```
#include <stdio.h>
#define MAX 5 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, element, position, i;

    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);

    // Read existing elements
    printf("Enter %d elements:\n", currentSize);
    for (i = 0; i < currentSize; i++)
    {
        scanf("%d", &list[i]);
    }

    // Check whether the list is full
    if (currentSize == MAX)
    {
        printf("List is full. Can't Insert\n");
        return 0;
    }
}
```

```
// Read new element to be inserted
printf("Enter element to insert: ");
scanf("%d", &element);

// Get position to insert (end of the list)
position = currentSize + 1;

// Insert element at the end
list[position - 1] = element;

// Increase current size
currentSize = currentSize + 1;

// Display updated list
printf("List after insertion:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}

return 0;
}
```

//C Program to Delete Element from End of the LIST

```
#include <stdio.h>
#define MAX 5 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, position, i;

    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);

    // Check whether the list is empty
    if (currentSize == 0)
    {
        printf("List is Empty. Can't Delete\n");
        return 0;
    }
```

```
// Read elements of the list
printf("Enter %d elements:\n", currentSize);
for (i = 0; i < currentSize; i++)
{
    scanf("%d", &list[i]);
}

// Get position of element to delete (end position)
position = currentSize;

// Delete element from the end
// (No shifting required for deletion at end)

// Decrease current size
currentSize = currentSize - 1;

// Display list after deletion
printf("List after deletion:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}
return 0;
}
```

```
#include <stdio.h>
#define MAX 5 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, element, i;

    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);

    // Read elements of the list
    printf("Enter %d elements:\n", currentSize);
    for (i = 0; i < currentSize; i++)
    {
        scanf("%d", &list[i]);
    }

    // Check whether the list is full
    if (currentSize == MAX)
    {
        printf("List is full. Can't Insert\n");
        return 0;
    }
}
```

//C Program to Insert Element to front of the LIST

```
// Read element to be inserted
printf("Enter element to insert at front: ");
scanf("%d", &element);

// Free the first position by shifting elements one position forward
for (i = currentSize - 1; i >= 0; i--)
{
    list[i + 1] = list[i];
}

// Insert element at the first position
list[0] = element;

// Increase current size
currentSize = currentSize + 1;

// Display updated list
printf("List after insertion at front:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}
return 0;
}
```

// C Program to Delete Element from front of the LIST

```
#include <stdio.h>
```

```
#define MAX 5 // Maximum size of the list
```

```
int main()
```

```
{
```

```
    int list[MAX];
```

```
    int currentSize, i;
```

```
    // Read current size of the list
```

```
    printf("Enter current size of the list: ");
```

```
    scanf("%d", &currentSize);
```

```
    // Check whether the list is empty
```

```
    if (currentSize == 0)
```

```
    {
```

```
        printf("List is Empty. Can't Delete\n");
```

```
        return 0;
```

```
    }
```

```
// Read elements of the list
printf("Enter %d elements:\n", currentSize);
for (i = 0; i < currentSize; i++)
{
    scanf("%d", &list[i]);
}

// Move all elements one position backward
for (i = 1; i < currentSize; i++)
{
    list[i - 1] = list[i];
}

// Decrease current size
currentSize = currentSize - 1;

// Display list after deletion
printf("List after deletion from front:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}
return 0;
}
```

//C program to Insert Element to nth Position of the LIST

```
#include <stdio.h>
#define MAX 10 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, element, position, i;
    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);
    // Check whether the list is full
    if (currentSize == MAX)
    {
        printf("List is full. Can't Insert\n");
        return 0;
    }
    // Read elements if list is not empty
    if (currentSize > 0)
    {
        printf("Enter %d elements:\n", currentSize);
        for (i = 0; i < currentSize; i++)
        {
            scanf("%d", &list[i]);
        }
    }
}
```

```
// Read position (n) to insert
printf("Enter the position to insert: ");
scanf("%d", &position);
// Read element to insert
printf("Enter element to insert: ");
scanf("%d", &element);
// If list is empty, insert at position 1
if (currentSize == 0 && position == 1)
{
    list[0] = element;
    currentSize = currentSize + 1;
}
// If position is greater than current size + 1
else if (position > currentSize + 1)
{
    printf("nth Position Not available in List\n");
    return 0;
}
```

```
else
{
    // Free the nth position by shifting elements forward
    for (i = currentSize - 1; i >= position - 1; i--)
    {
        list[i + 1] = list[i];
    }
    // Insert element at nth position
    list[position - 1] = element;

    // Increase current size
    currentSize = currentSize + 1;
}

// Display updated list
printf("List after insertion:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}

return 0;
}
```

//C Program to Delete Element from nth Position of the LIST

```
#include <stdio.h>
#define MAX 10 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, position, i;

    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);

    // Check whether the list is empty
    if (currentSize == 0)
    {
        printf("List is Empty.\n");
        return 0;
    }

    // Read elements of the list
    printf("Enter %d elements:\n", currentSize);
    for (i = 0; i < currentSize; i++)
    {
        scanf("%d", &list[i]);
    }
}
```

```
// Read nth position to delete
printf("Enter the position to delete: ");
scanf("%d", &position);

// If nth position is not available
if (position > currentSize || position < 1)
{
    printf("nth Position Not available in List\n");
    return 0;
}

// If nth position is the last position
if (position == currentSize)
{
    // Delete last element (no shifting required)
    currentSize = currentSize - 1;
}
```

```
else
{
    // Move elements one position backward (from n+1 to current size)
    for (i = position; i < currentSize; i++)
    {
        list[i - 1] = list[i];
    }

    // Decrease current size
    currentSize = currentSize - 1;
}

// Display list after deletion
printf("List after deletion:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}

return 0;
}
```

// C Program to Search Element in the LIST

```
#include <stdio.h>
#define MAX 10 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, searchElement;
    int i, position, found = 0;

    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);

    // Check whether the list is empty
    if (currentSize == 0)
    {
        printf("List is Empty\n");
        return 0;
    }
    // Read elements of the list
    printf("Enter %d elements:\n", currentSize);
    for (i = 0; i < currentSize; i++)
    {
        scanf("%d", &list[i]);
    }
}
```

```
// Read element to search
printf("Enter element to search: ");
scanf("%d", &searchElement);

// Last Position = Current Size
int lastPosition = currentSize;

// Search from Position 1 to Last Position
for (position = 1; position <= lastPosition; position++)
{
    if (list[position - 1] == searchElement)
    {
        printf("Search Element available in Position %d\n", position);
        found = 1;
        break;
    }
}
// If element not found
if (found == 0)
{
    printf("Search Element not available in the List\n");
}
return 0;
}
```

// C Program to Print the Elements in the LIST

```
#include <stdio.h>
```

```
#define MAX 10 // Maximum size of the list
```

```
int main()
```

```
{
```

```
    int list[MAX];
```

```
    int currentSize;
```

```
    int position;
```

```
    // Read current size of the list
```

```
    printf("Enter current size of the list: ");
```

```
    scanf("%d", &currentSize);
```

```
    // Check whether the list is empty
```

```
    if (currentSize == 0)
```

```
    {
```

```
        printf("List is Empty\n");
```

```
        return 0;
```

```
    }
```

```
// Read elements of the list
printf("Enter %d elements:\n", currentSize);
for (position = 1; position <= currentSize; position++)
{
    scanf("%d", &list[position - 1]);
}

// Last Position = Current Size
int lastPosition = currentSize;

// Print elements with position
printf("Position\tElement\n");
for (position = 1; position <= lastPosition; position++)
{
    printf("%d\t\t%d\n", position, list[position - 1]);
}

return 0;
}
```

****** END OF Module-1 Additional Information ******

Dr Dattatreya P Mankame